



Data Engineering Pipeline

Data Engineering / ETL-Architektur

Autor	Max Mustermann
Thema	Data Engineering / ETL-Architektur
Version	v1.0
Datum	12. Mai 2026

Erstellt mit md_to_pdf.py

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Überblick	4
1.1 Systemübersicht	4
1.2 Datenfluss	4
1.3 Verantwortlichkeiten	4
2. Extraktion	5
2.1 Quellsysteme	5
2.2 Python-Extractor	5
2.3 ERP-Extraktion via JDBC	6
3. Transformation mit dbt	7
3.1 Modell-Struktur	7
3.2 Staging-Modell: Kunden	7
3.3 Mart: Täglicher Umsatz	8
3.4 Intermediate-Modell: Kundenbestellungen	9
4. Orchestrierung mit Airflow	10
4.1 DAG-Konfiguration	10
4.2 DAG-Abhängigkeiten	11
4.3 Alerting & Benachrichtigungen	12
5. Infrastruktur & Konfiguration	12
5.1 dbt-Profil	12
5.2 Deploy-Pipeline	12
5.3 Monitoring-Konfiguration	13
6. Datenqualität & Tests	14
6.1 dbt-Tests	14
6.2 Datenqualitäts-Metriken	14
6.3 Fachlicher Kreuzvalidierungs-Test	15
7. Fehlerbehandlung & Recovery	15
7.1 Standard Recovery-Prozedur	15
7.2 Rollback DWH-Tabellen	16
7.3 Fehlerklassifikation	16
8. Kennzahlen & Performance	18
8.1 Laufzeiten (Durchschnitt letzte 30 Tage)	18
8.2 Performance-Tuning Python	18
8.3 SQL-Optimierung: Window Functions	18
8.4 Ressourcenverbrauch	19
9. Betrieb & Wartung	19
9.1 Regelmäßige Wartungsaufgaben	19
9.2 Dependency-Updates	19
9.3 Kapazitätsplanung	20

10. Glossar 20

1. Überblick

Dieses Dokument beschreibt die End-to-End Datenpipeline für die tägliche Verarbeitung von Transaktions- und Kundendaten. Die Pipeline wurde im Rahmen der konzernweiten Datenstrategie 2025 aufgebaut und löst das bisherige nächtliche Batch-Skript auf Basis von Shell und cron ab. Ziel war eine vollständig automatisierte, überwachte und wiederherstellbare Verarbeitungsstrecke, die auch bei Teilausfällen einzelner Quellsysteme stabil bleibt.

Die Pipeline verarbeitet täglich ca. **2,5 Mio. Datensätze** aus drei Quellsystemen und stellt aggregierte Reports im Data Warehouse bereit. Durch die Einführung von dbt als Transformations-Layer konnte die durchschnittliche Verarbeitungszeit gegenüber dem alten System um **38 %** reduziert werden. Gleichzeitig stieg die Test-Abdeckung von 0 auf über 120 automatisierte Datenqualitätsprüfungen.

■ HINWEIS

Alle Zeitangaben beziehen sich auf die Zeitzone Europe/Berlin (CET/CEST). Cron-Jobs laufen auf dem internen Scheduler-Server (scheduler01.internal).

1.1 Systemübersicht

Komponente	Technologie	Version	Verantwortlich
Orchestrierung	Apache Airflow	2.9	Data Engineering
Datenbank (DWH)	PostgreSQL	15.4	Data Engineering
Staging-Layer	MinIO (S3-kompatibel)	2024.x	Infrastructure
Transformation	dbt	1.7	Analytics
Monitoring	Grafana + Prometheus	10.x	Infrastructure

1.2 Datenfluss

Die Pipeline besteht aus vier klar abgegrenzten Phasen. Jede Phase ist eigenständig testbar und kann im Fehlerfall unabhängig neu gestartet werden:

- **Extraktion** – Quelldaten werden via REST-API und JDBC abgerufen und als Rohdaten im Staging-Layer abgelegt
- **Staging** – Rohdaten landen als unveränderliche Parquet-Dateien in MinIO; keine Transformation in dieser Phase
- **Transformation** – dbt-Modelle bereinigen, vereinheitlichen und aggregieren die Daten in mehreren Schichten (Staging → Intermediate → Marts)
- **Laden** – Finale Mart-Tabellen werden atomar ins DWH geschrieben und stehen den nachgelagerten BI-Tools zur Verfügung

Die Phasen 1 und 2 laufen parallel für alle drei Quellsysteme. Phase 3 startet erst, wenn alle Extraktionen erfolgreich abgeschlossen sind (Airflow-Dependency-Graph). Phase 4 ist in dbt bereits enthalten und wird nicht separat gesteuert.

1.3 Verantwortlichkeiten

Bereich	Team	Ansprechpartner
Pipeline-Entwicklung	Data Engineering	Max Mustermann
DWH-Schema	Data Engineering	Max Mustermann
dbt-Modelle (Marts)	Analytics	Jana Müller

Bereich	Team	Ansprechpartner
Infrastruktur / MiniIO	Infrastructure	Tobias Kranz
Monitoring & Alerts	Infrastructure	Tobias Kranz
Fachliche Anforderungen	Business Intelligence	Sabine Hoffmann

2. Extraktion

2.1 Quellsysteme

Die Pipeline zieht Daten aus drei Systemen. Jedes System hat eigene Verbindungsparameter, Authentifizierungsmechanismen und Eigenheiten bei der Delta-Erkennung:

- **ERP-System** (SAP S/4HANA) – Transaktionsdaten via JDBC; Delta-Erkennung über Belegdatum ERDAT
- **CRM-System** (Salesforce) – Kundendaten via REST-API; Delta-Erkennung über `modified_since`-Parameter
- **Web Analytics** (Matomo) – Klick- und Sessiondaten via API; täglicher Volllauf da keine zuverlässige Delta-Logik verfügbar

Die Verbindungsparameter aller Systeme werden ausschließlich über Umgebungsvariablen übergeben und sind nicht im Code oder in Konfigurationsdateien hinterlegt. Secrets werden über HashiCorp Vault verwaltet und beim Start der Airflow-Worker zur Laufzeit injiziert.

■ WARNUNG

Das ERP-System hat ein tägliches Wartungsfenster von 02:00–02:30 Uhr. Extraktion darf nicht in diesem Zeitraum gestartet werden.

2.2 Python-Extractor

PYTHON

```

import logging
from datetime import datetime, timedelta
from pathlib import Path
import requests
import pandas as pd

logger = logging.getLogger(__name__)

class CRMExtractor:
    """Lädt Kundendaten aus der Salesforce REST-API."""

    BASE_URL = "https://api.salesforce.internal/v2"

    def __init__(self, token: str, batch_size: int = 1000):
        self.token = token
        self.batch_size = batch_size
        self.session = requests.Session()
        self.session.headers.update({"Authorization": f"Bearer {token}"})

    def fetch_delta(self, since: datetime) -> pd.DataFrame:
        """Holt alle geänderten Datensätze seit 'since'."""
        records = []
        offset = 0

        while True:
            params = {
                "modified_since": since.isoformat(),
                "limit": self.batch_size,
                "offset": offset,
            }
            resp = self.session.get(f"{self.BASE_URL}/customers", params=params)
            resp.raise_for_status()
            batch = resp.json().get("data", [])
            if not batch:
                break
            records.extend(batch)
            offset += len(batch)
            logger.info(f"Geladen: {offset} Datensätze")

        df = pd.DataFrame(records)
        logger.info(f"Extraktion abgeschlossen: {len(df)} Zeilen")
        return df

    def save_to_staging(self, df: pd.DataFrame, staging_path: Path) -> None:
        """Speichert den DataFrame als Parquet im Staging-Bereich."""
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        out = staging_path / f"crm_customers_{ts}.parquet"
        df.to_parquet(out, index=False, compression="snappy")
        logger.info(f"Staging: {out}")

```

2.3 ERP-Extraktion via JDBC

Die JDBC-Verbindung zum ERP-System wird über einen dedizierten HikariCP Connection Pool mit maximal 4 parallelen Verbindungen verwaltet. Der Pool wird beim Start des Airflow-Workers initialisiert und für die gesamte Laufzeit gehalten, um Verbindungsaufbau-Overhead zu minimieren.

Die Extraktion liest primär aus der VBRK-Tabelle (Fakturen-Kopfdaten) und verknüpft diese mit VBRP (Positionsdaten). Beide Tabellen haben in der Produktivumgebung jeweils ca. 180 Mio. Datensätze. Durch den gesetzten Index auf ERDAT ist die Delta-Abfrage trotz Tabellengröße performant (Laufzeit < 3 min).

SQL

```
-- Delta-Extraktion Fakturen (letzter Tag)
SELECT
    VBELN                AS invoice_id,
    KUNNR                AS customer_id,
    ERDAT                AS created_date,
    FKDAT                AS billing_date,
    WAERK                AS currency,
    NETWR                AS net_amount,
    MWSBP                AS tax_amount,
    NETWR + MWSBP        AS gross_amount,
    VKORG                AS sales_org
FROM VBRK
WHERE ERDAT = CURRENT_DATE - INTERVAL '1 day'
      AND FKART IN ('F2', 'G2', 'L2')
      AND RFBSK = 'C'
ORDER BY ERDAT, VBELN;
```

3. Transformation mit dbt

dbt (Data Build Tool) ist das zentrale Transformations-Framework der Pipeline. Alle Transformationslogik ist in versioniertem SQL definiert und über Git nachvollziehbar. dbt übernimmt dabei nicht nur die Ausführung der SQL-Modelle, sondern auch Abhängigkeitsauflösung, automatische Tests und Dokumentationsgenerierung.

Das Deployment unterscheidet zwei Zielumgebungen: dev für Entwicklung und Testing auf einem dedizierten Entwicklungs-DWH, und prod für den produktiven Nachlauf. Änderungen an Modellen durchlaufen immer erst dev mit einem vollständigen dbt test-Lauf bevor sie in prod deployt werden.

3.1 Modell-Struktur

```
models/
├── staging/
│   ├── stg_crm_customers.sql
│   ├── stg_erp_invoices.sql
│   └── stg_web_sessions.sql
├── intermediate/
│   ├── int_customer_orders.sql
│   └── int_revenue_daily.sql
└── marts/
    ├── mart_sales_daily.sql
    └── mart_customer_360.sql
```

3.2 Staging-Modell: Kunden

SQL

```
-- models/staging/stg_crm_customers.sql
{{ config(materialized='view') }}

WITH source AS (
    SELECT * FROM {{ source('crm_raw', 'customers') }}
),

cleaned AS (
    SELECT
        CAST(customer_id AS VARCHAR(20)) AS customer_id,
        TRIM(UPPER(first_name)) AS first_name,
        TRIM(UPPER(last_name)) AS last_name,
        LOWER(email) AS email,
        CAST(created_at AS TIMESTAMP) AS created_at,
        COALESCE(country_code, 'DE') AS country_code,
        CASE
            WHEN status = 'A' THEN 'active'
            WHEN status = 'I' THEN 'inactive'
            ELSE 'unknown'
        END AS customer_status,
        _loaded_at AS dbt_loaded_at
    FROM source
    WHERE customer_id IS NOT NULL
)

SELECT * FROM cleaned
```

3.3 Mart: Täglicher Umsatz

SQL

```
-- models/marts/mart_sales_daily.sql
{{ config(
    materialized = 'incremental',
    unique_key   = ['report_date', 'sales_org'],
    on_schema_change = 'sync_all_columns'
) }}

WITH daily_revenue AS (
    SELECT
        billing_date                AS report_date,
        sales_org,
        COUNT(DISTINCT invoice_id) AS invoice_count,
        COUNT(DISTINCT customer_id) AS unique_customers,
        SUM(net_amount)             AS total_net,
        SUM(gross_amount)           AS total_gross,
        AVG(gross_amount)           AS avg_invoice_value,
        MAX(gross_amount)           AS max_invoice_value
    FROM {{ ref('stg_erp_invoices') }}
    {% if is_incremental() %}
        WHERE billing_date >= (SELECT MAX(report_date) FROM {{ this }})
    {% endif %}
    GROUP BY billing_date, sales_org
),

with_rank AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY sales_org
            ORDER BY total_gross DESC
        ) AS revenue_rank
    FROM daily_revenue
)

SELECT * FROM with_rank
```

✓ TIPP

Bei `incremental`-Modellen immer `--full-refresh` ausführen, wenn sich die Schema-Struktur geändert hat: `dbt run --select mart\_sales\_daily --full-refresh`

3.4 Intermediate-Modell: Kundenbestellungen

Das Intermediate-Modell verbindet CRM- und ERP-Daten auf Kundenebene. Es dient als Basis für mehrere Mart-Modelle und wird als `table` materialisiert, da es von mehreren nachgelagerten Modellen referenziert wird und eine eigene Berechnungslogik enthält, die nicht wiederholt werden soll.

SQL

```
-- models/intermediate/int_customer_orders.sql
{{ config(materialized='table') }}

WITH customers AS (
    SELECT * FROM {{ ref('stg_crm_customers') }}
    WHERE customer_status = 'active'
),

invoices AS (
    SELECT * FROM {{ ref('stg_erp_invoices') }}
    WHERE billing_date >= CURRENT_DATE - INTERVAL '24 months'
),

joined AS (
    SELECT
        c.customer_id,
        c.first_name || ' ' || c.last_name      AS customer_name,
        c.country_code,
        c.created_at                          AS customer_since,
        COUNT(i.invoice_id)                   AS total_orders,
        SUM(i.gross_amount)                    AS lifetime_value,
        MAX(i.billing_date)                    AS last_order_date,
        CURRENT_DATE - MAX(i.billing_date)     AS days_since_last_order
    FROM customers c
    LEFT JOIN invoices i USING (customer_id)
    GROUP BY
        c.customer_id, c.first_name, c.last_name,
        c.country_code, c.created_at
)

SELECT
    *,
    CASE
        WHEN lifetime_value >= 10000 THEN 'A'
        WHEN lifetime_value >= 2500  THEN 'B'
        WHEN lifetime_value >= 500   THEN 'C'
        ELSE                          THEN 'D'
    END
    AS customer_segment
FROM joined
```

4. Orchestrierung mit Airflow

Apache Airflow dient als zentraler Orchestrator der gesamten Pipeline. Alle DAGs sind als Python-Code in einem dedizierten Git-Repository verwaltet und werden über CI/CD automatisch auf den Airflow-Server deployt. Die Airflow-Instanz läuft auf Kubernetes (3 Worker-Pods, autoscaling bis 8) und ist über einen internen Load Balancer erreichbar.

4.1 DAG-Konfiguration

PYTHON

```

from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.utils.dates import days_ago

default_args = {
    "owner": "data-engineering",
    "depends_on_past": False,
    "email_on_failure": True,
    "email": ["de-alerts@company.internal"],
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id = "daily_etl_pipeline",
    default_args = default_args,
    description = "Tägliche ETL-Pipeline: ERP + CRM → DWH",
    schedule_interval = "0 3 * * 1-6", # Mo-Sa um 03:00 Uhr
    start_date = datetime(2026, 1, 1),
    catchup = False,
    tags = ["etl", "production"],
) as dag:

    extract_crm = PythonOperator(
        task_id = "extract_crm_customers",
        python_callable = run_crm_extraction,
    )

    extract_erp = PythonOperator(
        task_id = "extract_erp_invoices",
        python_callable = run_erp_extraction,
    )

    run_dbt = BashOperator(
        task_id = "run_dbt_models",
        bash_command = "cd /opt/dbt && dbt run --profiles-dir . --target prod",
    )

    run_tests = BashOperator(
        task_id = "run_dbt_tests",
        bash_command = "cd /opt/dbt && dbt test --profiles-dir . --target prod",
    )

    [extract_crm, extract_erp] >> run_dbt >> run_tests

```

4.2 DAG-Abhängigkeiten

Task	Upstream	SLA	Retry
extract ^{crm} customers	–	04:00	2×
extract ^{erp} invoices	–	04:00	2×
run ^{dbt} models	extract ^{crm} , extract ^{erp}	05:00	1×
run ^{dbt} tests	run ^{dbt} models	05:30	0×

! WICHTIG

SLA-Verletzungen werden automatisch per E-Mail an das DE-Team gemeldet. Zusätzlich wird ein Alert in Grafana ausgelöst (Panel: `ETL Pipeline SLA`).

4.3 Alerting & Benachrichtigungen

Airflow ist mit dem internen SMTP-Server verbunden. Bei Task-Failure erhält das DE-Team innerhalb von 2 Minuten eine E-Mail mit dem vollständigen Stack-Trace und einem direkten Link auf den fehlgeschlagenen Task im Airflow-UI. Zusätzlich ist ein Slack-Webhook konfiguriert, der in den Kanal #de-alerts postet.

Folgende Ereignisse lösen automatisch Benachrichtigungen aus:

- **Task Failed** – sofortige Benachrichtigung per E-Mail und Slack
- **Task Retry** – Slack-Nachricht (kein E-Mail um Alert-Fatigue zu vermeiden)
- **SLA Missed** – E-Mail an DE-Team und Führungskraft
- **DAG Success** – tägliche Zusammenfassung per E-Mail (06:00 Uhr)

5. Infrastruktur & Konfiguration

5.1 dbt-Profil

YAML

```
# profiles.yml
data_engineering:
  target: prod
  outputs:
    dev:
      type: postgres
      host: db-dev.internal
      port: 5432
      user: "{{ env_var('DBT_DEV_USER') }}"
      password: "{{ env_var('DBT_DEV_PASS') }}"
      dbname: analytics_dev
      schema: dbt_dev
      threads: 4

    prod:
      type: postgres
      host: db-prod.internal
      port: 5432
      user: "{{ env_var('DBT_PROD_USER') }}"
      password: "{{ env_var('DBT_PROD_PASS') }}"
      dbname: analytics_prod
      schema: marts
      threads: 8
      keepalives_idle: 300
```

5.2 Deploy-Pipeline

BASH

```
#!/usr/bin/env bash
set -euo pipefail

ENVIRONMENT=${1:-dev}
DBT_DIR="/opt/dbt"

echo "=== Deploying dbt models to: ${ENVIRONMENT} ==="

# Abhängigkeiten installieren
pip install -r requirements.txt --quiet

# Verbindung testen
dbt debug --profiles-dir "${DBT_DIR}" --target "${ENVIRONMENT}"

# Modelle ausführen
dbt run \
  --profiles-dir "${DBT_DIR}" \
  --target "${ENVIRONMENT}" \
  --exclude "tag:experimental"

# Tests ausführen
dbt test \
  --profiles-dir "${DBT_DIR}" \
  --target "${ENVIRONMENT}"

echo "=== Deployment abgeschlossen ==="
```

5.3 Monitoring-Konfiguration

JSON

```
{
  "alert_rules": [
    {
      "name": "ETL Pipeline Failed",
      "condition": "dag_run_state == 'failed'",
      "severity": "critical",
      "channels": ["email", "slack"],
      "cooldown_minutes": 30
    },
    {
      "name": "SLA Breach",
      "condition": "task_duration_seconds > 7200",
      "severity": "warning",
      "channels": ["email"],
      "cooldown_minutes": 60
    },
    {
      "name": "Row Count Anomaly",
      "condition": "row_count < expected_rows * 0.9",
      "severity": "warning",
      "channels": ["slack"],
      "cooldown_minutes": 120
    }
  ],
  "dashboards": {
    "etl_overview": "grafana.internal/d/etl-overview",
    "dbt_runs": "grafana.internal/d/dbt-runs"
  }
}
```

6. Datenqualität & Tests

Datenqualität ist eine gemeinsame Verantwortung von Data Engineering und Analytics. Das DE-Team stellt sicher, dass die Rohdaten vollständig und technisch korrekt ins DWH gelangen. Das Analytics-Team verantwortet die fachliche Korrektheit der Transformationen und Aggregationen. Beide Teams definieren gemeinsam die Qualitätsschwellwerte und Eskalationsprozesse.

Die Qualitätsprüfungen gliedern sich in drei Ebenen:

- **Technische Tests** (dbt built-in) – *notnull, unique, acceptedvalues*, referential integrity
- **Statistische Tests** (dbt-expectations) – Wertebereich, Verteilung, Row-Count-Anomalien
- **Fachliche Tests** (custom SQL) – Plausibilitätsprüfungen, Kreuzvalidierung zwischen Quellsystemen

6.1 dbt-Tests

YAML

```
# models/staging/schema.yml
version: 2

models:
  - name: stg_crm_customers
    description: "Bereinigte Kundendaten aus dem CRM-System"
    columns:
      - name: customer_id
        tests:
          - not_null
          - unique
      - name: email
        tests:
          - not_null
      - name: customer_status
        tests:
          - accepted_values:
              values: ['active', 'inactive', 'unknown']

  - name: stg_erp_invoices
    description: "Fakturadaten aus SAP ERP"
    columns:
      - name: invoice_id
        tests:
          - not_null
          - unique
      - name: net_amount
        tests:
          - not_null
          - dbt_expectations.expect_column_values_to_be_between:
              min_value: 0
              max_value: 1000000
```

6.2 Datenqualitäts-Metriken

Metrik	Schwellwert	Aktuell (letzte 7 Tage)	Status
Null-Rate customer_id	< 0,01 %	0,003 %	OK
Duplicate-Rate invoices	< 0,001 %	0,000 %	OK
Schema-Drift-Events	0 / Woche	0	OK
Row-Count-Abweichung	< 5 %	1,2 %	OK
Test-Erfolgsrate (dbt)	100 %	99,8 %	WARN

■ WARNUNG

Die Test-Erfolgsrate liegt leicht unter 100 %. Ursache: 3 fehlgeschlagene
`accepted\_values`-Tests bei `customer\_status` durch neu eingeführten Status-Code `P`
(Prospect) im CRM. Fix in Arbeit (Ticket #DE-418).

6.3 Fachlicher Kreuzvalidierungs-Test

SQL

```
-- tests/cross_check_revenue_crm_erp.sql
-- Prüft: Kunden im Umsatz-Mart müssen im CRM bekannt sein
SELECT
    i.customer_id,
    SUM(i.gross_amount) AS umsatz_ohne_crm_eintrag
FROM marts.mart_sales_daily i
LEFT JOIN marts.mart_customer_360 c USING (customer_id)
WHERE c.customer_id IS NULL
    AND i.report_date >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY i.customer_id
HAVING SUM(i.gross_amount) > 0
```

Wenn dieser Test Zeilen zurückgibt, existieren Umsätze für Kunden, die im CRM nicht vorhanden sind. Das deutet entweder auf einen Sync-Fehler im CRM oder auf Direktbuchungen im ERP hin, die ohne CRM-Kundeneintrag entstanden sind. In beiden Fällen ist eine manuelle Prüfung erforderlich.

7. Fehlerbehandlung & Recovery

Die Pipeline ist nach dem Prinzip **Fail Fast, Recover Safe** aufgebaut. Tasks schlagen sofort fehl wenn ein Fehler auftritt – es gibt kein stilles Ignorieren von Fehlern. Jeder Fehler wird protokolliert, gemeldet und muss aktiv quittiert werden.

7.1 Standard Recovery-Prozedur

BASH

```
#!/usr/bin/env bash
# recovery.sh – Neustart einer fehlgeschlagenen Pipeline
set -euo pipefail

DAG_ID="daily_etl_pipeline"
RUN_DATE=${1:-$(date +%Y-%m-%d)}

echo "Recovery für DAG: ${DAG_ID}, Datum: ${RUN_DATE}"

# 1. Fehlerhafte Run-Instanz löschen
airflow dags delete-run \
  --dag-id "${DAG_ID}" \
  --run-id "scheduled__${RUN_DATE}T03:00:00+00:00"

# 2. Staging-Dateien bereinigen
python3 cleanup_staging.py --date "${RUN_DATE}"

# 3. DAG manuell triggern
airflow dags trigger \
  --dag-id "${DAG_ID}" \
  --conf '{"run_date": "${RUN_DATE}"}'

echo "Recovery-Trigger gesendet."
```

7.2 Rollback DWH-Tabellen

SQL

```
-- Rollback: Tageswerte aus Mart entfernen und neu laden
BEGIN;

DELETE FROM marts.mart_sales_daily
WHERE report_date = CURRENT_DATE - INTERVAL '1 day';

DELETE FROM marts.mart_customer_360
WHERE snapshot_date = CURRENT_DATE - INTERVAL '1 day';

-- Nach erfolgreichem dbt-Run:
-- COMMIT;
-- Im Fehlerfall:
-- ROLLBACK;
```

■ HINWEIS

Der Rollback muss immer manuell durchgeführt und nach dem erneuten dbt-Run mit `COMMIT` abgeschlossen werden. Niemals ohne anschließenden dbt-Run committen.

7.3 Fehlerklassifikation

Nicht alle Fehler haben die gleiche Kritikalität. Die folgende Tabelle beschreibt die drei Fehlerklassen und die jeweilige Reaktion:

Klasse	Beispiel	Reaktion	Eskalation
P1	Gesamter DAG ausgefallen	Sofort: Oncall + DE-Lead	Innerhalb 15 min
P2	Einzelner Task fehlgeschlagen, SLA ok	Recovery-Skript, kein Oncall	Bei > 2× pro Woche
P3	Datenqualitäts-Warnung (kein Abbruch)	Ticket erstellen, nächster Werktag	Bei > 5× pro Monat

■ HINWEIS

P1-Vorfälle werden in einer separaten Postmortem-Datei dokumentiert. Die Vorlage liegt unter ``/runbooks/postmortem_template.md`` im DE-Repository.

8. Kennzahlen & Performance

8.1 Laufzeiten (Durchschnitt letzte 30 Tage)

Task	Ø Laufzeit	Min	Max	P95
extractcrmcustomers	4 min 12 s	3:45	8:30	7:15
extracterpinvoices	11 min 5 s	9:10	22:40	18:30
rundbtmodels	18 min 30 s	15:20	35:10	28:45
rundbttests	6 min 20 s	5:00	14:20	11:00
Gesamt	40 min	33:15	1:20:40	1:05:30

8.2 Performance-Tuning Python

PYTHON

```
# Optimierte Parquet-Partition-Strategie
import pyarrow as pa
import pyarrow.parquet as pq
from pathlib import Path

def write_partitioned_parquet(
    df: pd.DataFrame,
    base_path: Path,
    partition_cols: list[str],
) -> None:
    """Schreibt Daten partitioniert nach Datum und Region."""
    table = pa.Table.from_pandas(df, preserve_index=False)
    pq.write_to_dataset(
        table,
        root_path = str(base_path),
        partition_cols = partition_cols,
        compression = "snappy",
        use_deprecated_int96_timestamps = False,
    )

# Aufruf
write_partitioned_parquet(
    df = df_invoices,
    base_path = Path("/mnt/staging/erp/invoices"),
    partition_cols = ["billing_date", "sales_org"],
)
```

8.3 SQL-Optimierung: Window Functions

SQL

```
-- Kundenranking nach Umsatz mit LAG/LEAD für Trendberechnung
WITH customer_monthly AS (
  SELECT
    customer_id,
    DATE_TRUNC('month', billing_date)      AS month,
    SUM(gross_amount)                     AS monthly_revenue
  FROM mart_sales_daily
  WHERE billing_date >= CURRENT_DATE - INTERVAL '12 months'
  GROUP BY customer_id, DATE_TRUNC('month', billing_date)
),

with_trend AS (
  SELECT
    customer_id,
    month,
    monthly_revenue,
    LAG(monthly_revenue) OVER w           AS prev_month,
    LEAD(monthly_revenue) OVER w          AS next_month,
    RANK() OVER (PARTITION BY month
                 ORDER BY monthly_revenue DESC) AS rank
  FROM customer_monthly
  WINDOW w AS (PARTITION BY customer_id ORDER BY month)
)

SELECT
  customer_id,
  month,
  monthly_revenue,
  ROUND(
    (monthly_revenue - prev_month) / NULLIF(prev_month, 0) * 100, 2
  ) AS growth_pct,
  rank
FROM with_trend
WHERE rank <= 100
ORDER BY month DESC, rank ASC;
```

8.4 Ressourcenverbrauch

Der durchschnittliche Ressourcenverbrauch pro Nachlauf liegt bei ca. 12 GB RAM (Peak: 22 GB während der dbt-Transformation) und belegt 4 der 8 verfügbaren DWH-Verbindungen gleichzeitig. Die MinIO-Staging-Ablage wächst täglich um ca. 3,2 GB (Parquet, Snappy-komprimiert). Rohdaten werden nach 30 Tagen automatisch gelöscht, aggregierte Mart-Daten werden unbefristet aufbewahrt.

9. Betrieb & Wartung

9.1 Regelmäßige Wartungsaufgaben

Folgende Aufgaben fallen regelmäßig an und sind im DE-Team aufgeteilt:

- **Täglich** – Alert-Log prüfen, offene P2/P3-Tickets sichten
- **Wöchentlich** – dbt-Dokumentation aktualisieren (dbt docs generate), Laufzeit-Trends in Grafana prüfen
- **Monatlich** – Staging-Storage auditieren, veraltete Modelle identifizieren, Abhängigkeiten aktualisieren
- **Quartalsweise** – Vollständiges dbt build --full-refresh auf dev, Kapazitätsplanung DWH

9.2 Dependency-Updates

BASH

```
# Alle Python-Abhängigkeiten auf neue Versionen prüfen
pip list --outdated

# dbt-Pakete aktualisieren (packages.yml)
dbt deps --upgrade

# Airflow-Provider aktualisieren
pip install apache-airflow-providers-postgres \
            apache-airflow-providers-amazon \
            --upgrade
```

■ WARNUNG

Airflow-Major-Version-Updates (z. B. 2.x → 3.x) erfordern eine vollständige Testdurchführung auf `dev` inklusive aller DAGs. Nie direkt auf `prod` updaten.

9.3 Kapazitätsplanung

Das Datenwachstum beträgt derzeit ca. **12 % pro Quartal** (gemessen an der Zahl der täglichen Transaktionsdatensätze). Bei gleichbleibendem Wachstum ist in **Q3 2027** mit einer Verdoppelung des aktuellen Datenvolumens zu rechnen. Folgende Maßnahmen sind in Planung:

- **Partitionierung der Mart-Tabellen** nach report_date (Ticket #DE-441, geplant Q2 2026)
- **Archivierungsstrategie** für Transaktionsdaten älter als 3 Jahre (in Abstimmung mit Legal)
- **Evaluation columnar DWH** (DuckDB oder ClickHouse) als langfristige Alternative zu PostgreSQL

10. Glossar

Begriff	Beschreibung
DAG	Directed Acyclic Graph – Airflow-Workflow-Struktur
DWH	Data Warehouse – zentrales analytisches Datenbanksystem
dbt	Data Build Tool – SQL-Transformations-Framework
ELT	Extract → Load → Transform (Laden vor Transformation)
ETL	Extract → Transform → Load (Transformation vor Laden)
Incremental	dbt-Materialisierung, die nur neue/geänderte Daten verarbeitet
Mart	Data Mart – themenspezifische, aggregierte DWH-Schicht
Parquet	Spaltenorientiertes Binär-Dateiformat für Big Data
Staging	Zwischenspeicher für Rohdaten vor der Transformation
SLA	Service Level Agreement – vereinbarte Laufzeit-Grenze

HikariCP	Java-Connection-Pool-Bibliothek für JDBC-Verbindungen
Postmortem	strukturierte Fehleranalyse nach einem P1-Vorfall
Vault	HashiCorp Vault – zentrales Secret-Management

✓ TIPP

Die vollständige Systemdokumentation, ER-Diagramme und Runbooks sind im internen Confluence unter `Data Engineering > Pipeline Docs` verfügbar.